IBVAP — System Architecture & Data Pipeline

02. Complete IBVAP End-to-End Architecture

1. High-Level Data Flow Pipeline

```
Video Feed (data/video.mp4)
       ■
       ▼
YOLO11 Detection Engine (models/yolo11s.pt / src/main.py)
  ■■ Class: 0 (Person)
  ■■ Frame-by-frame Bounding Box Extraction
       ■
       ▼
ByteTrack Association Engine (bytetrack_day3.yaml)
  ■■ Kalman Filter Motion Prediction
  ■■ Hungarian Matching (High/Low Conf Thresholds)
  ■■ Persistent Track ID Assignment
       ■
       ▼
Restricted Zone Spatial Processor
  ■■ Polygon Boundaries Configuration
  ■■ Bounding Box Center Point-in-Polygon Check
       ■
       ▼
Temporal Validation & Event Generator
  ■■ Trigger: Person Bounding Box intersects Restricted Polygon
  ■■ Event Record: EVT-ID, Track ID, Frame, Timestamp, Risk ("HIGH")
       ■
       ▼
Digital Evidence Snapshot Generator
  ■■ High-Resolution Frame Grab at Breach Point
  ■■ File Output: output/evidence/EVT-XXX_track_Y_frame_Z.jpg
       ■
       ▼
Multi-Tier Database Layer (src/app.py / data_service.py)
  ■■ Tier 1: MongoDB Atlas / Supabase Cloud DB
  ■■ Tier 2: Local CSV Log (output/intrusion_log.csv)
  ■■ Tier 3: Hardcoded Seed Fallback Data
       ■
       ▼
Cryptographic Audit Logger (src/app.py)
  ■■ Continuous SHA-256 Hash Chaining across Event Strings
  ■■ Output File: output/secure_audit_log.csv
  ■■ Root Hash File: output/audit_root_hash.txt
       ■
       ▼
Local Blockchain-Style Audit Ledger (src/app.py)
  ■■ Block Genesis & Payload Assembly
  ■■ Output File: output/blockchain_ledger.json
       ■
       ▼
Streamlit Command Center UI (streamlit_app/app.py / src/app.py)
  ■■ Executive Command Center Dashboard
  ■■ Surveillance & Zone Visualizer
  ■■ Security Events Data Table & Inspector
  ■■ Digital Evidence High-Res Gallery
  ■■ Audit Log & Real-Time Integrity Verification
  ■■ Technical System Information
```

2. Stage-by-Stage Architectural Breakdown

Stage 1: Video Stream Ingestion

- **Technology:** OpenCV (`cv2.VideoCapture`)
- **File:** `src/main.py` / `src/app.py`
- **Input:** Surveillance video stream (`data/video.mp4` / `output/final_intrusion_video.mp4`)
- **Output:** Raw image frame arrays (NumPy arrays `(H, W, C)` in BGR format)
- **Failure Handling:** Returns `success == False`, cleanly releases video handle.

Stage 2: Object Detection

- **Technology:** Ultralytics YOLO11 (`yolo11s.pt` / `yolo11n.pt`)
- **File:** `src/main.py` (Line 5, Line 17)
- **Function:** `model.track(frame, persist=True, classes=[0])`
- **Input:** Single video frame
- **Output:** Bounding box coordinates `[x1, y1, x2, y2]`, confidence score `conf`, class label `0` (Person)
- **Failure Handling:** If model weights missing, raises `FileNotFoundError`.

Stage 3: Multi-Object Tracking

- **Technology:** ByteTrack (`bytetrack_day3.yaml`)
- **File:** Hyperparameters in `bytetrack_day3.yaml`
- **Input:** Frame detections + previous track states
- **Output:** Persistent numeric `track_id` assigned to each individual target.
- **Logic:** Two-stage matching (high confidence ≥ 0.25, low confidence ≥ 0.10), score fusion, track buffer of 60 frames.

Stage 4: Restricted Zone Analysis

- **Technology:** Polygon spatial intersection (Point-in-Polygon check)
- **File:** Evaluated in `Day2_tracking.ipynb` & visualized in `streamlit_app/pages/surveillance.py`
- **Input:** Person bounding box center point `(cx, cy)`, polygon vertices `[(x1,y1), (x2,y2), ...]`
- **Output:** Boolean flag (`is_inside_zone`) and event state (`ENTER`, `EXIT`, `INITIAL_INSIDE`)

Stage 5: Intrusion Event Generation & Evidence Capture

- **Technology:** OpenCV snapshot saving + Python dataclass payload creation
- **File:** `src/app.py` / `output/evidence/`
- **Input:** Confirmed breach frame, `track_id`, frame index, timestamp string
- **Output:** Snapshot `.jpg` saved to `output/evidence/` + `IntrusionEvent` dataclass instance.

Stage 6: Multi-Tier Data Persistence

- **Technology:** MongoDB PyMongo client / Supabase REST API / Pandas CSV writer
- **File:** `src/app.py` (Lines 83–380) / `streamlit_app/utils/data_service.py` (Lines 46–151)

- **Input:** `IntrusionEvent` instance
- **Output:** Persistent database record or CSV entry
- **Failure Handling:** Fallback chain: Cloud DB (MongoDB/Supabase) $\rightarrow$ Local CSV (`output/intrusion_log.csv`) $\rightarrow$ Seed Data.

Stage 7: SHA-256 Cryptographic Audit Chain

- **Technology:** Python `hashlib` (SHA-256)
- **File:** `src/app.py` (Lines 387–430)
- **Input:** Sequential event records + `previous_hash`
- **Output:** Hash-chained log `output/secure_audit_log.csv` and root hash `output/audit_root_hash.txt`

Stage 8: Local Blockchain-Style Audit Ledger

- **Technology:** JSON block linking with SHA-256 headers
- **File:** `src/app.py` (Lines 433–505) / `output/blockchain_ledger.json`
- **Input:** Verified audit records
- **Output:** Structured JSON ledger containing Genesis block (Index 0) and linked event blocks (Index 1..N).

Stage 9: Streamlit Web Command Center

- **Technology:** Streamlit 1.38+, Plotly, HTML/CSS custom components
- **File:** `streamlit_app/app.py` / `src/app.py`
- **Input:** Event collections, video path, audit files
- **Output:** Web UI with real-time integrity verification indicators.